

Aula 4 – Construtores e Destrutores em C#

Programação Orientada a Objetos
Professor: Eduardo Luan Pilonetto

Objetivos da Aula

- Compreender o conceito de construtores.
- Inicializar objetos corretamente.
- Conhecer sobrecarga de construtores.
- Entender inicialização automática.
- Conhecer destrutores.

Agenda da Aula

- Revisão rápida de classes e objetos
- Construtores
- Sobrecarga de construtores
- Inicialização automática
- Destrutores
- Exercícios práticos
- Desafio final

Revisão: Classes e Objetos

- Classe: modelo para criação de objetos.
- Objeto: instância de uma classe.
- Atributos representam características.
- Métodos representam comportamentos.

O que é um Construtor?

- Método especial executado automaticamente.
- Chamado ao criar um objeto.
- Serve para inicializar atributos.
- Possui o mesmo nome da classe.

Características dos Construtores

- Não possuem tipo de retorno.
- Podem receber parâmetros.
- Executados automaticamente.
- Podem ser sobrecarregados.

Exemplo Básico de Construtor

```
class Pessoa
{
    public string Nome;

    public Pessoa()
    {
        Nome = "Não informado";
    }
}
```

Criando Objetos

```
Pessoa p1 = new Pessoa();
```

```
Console.WriteLine(p1.Nome);
```


Construtor com Parâmetros

```
class Produto
{
    public string Nome;
    public double Preco;

    public Produto(string nome, double preco)
    {
        Nome = nome;
        Preco = preco;
    }
}
```

Criando Objetos com Parâmetros

```
Produto p = new Produto("Notebook", 3500);
```

```
Console.WriteLine(p.Nome);
```

```
Console.WriteLine(p.Preco);
```

Sobrecarga de Construtores

- Uma classe pode possuir vários construtores.
- Cada construtor deve possuir assinatura diferente.
- Facilita diferentes formas de inicialização.

Exemplo de Sobrecarga

```
class Aluno
{
    public string Nome;
    public int Idade;

    public Aluno()
    {
        Nome = "Sem nome";
    }

    public Aluno(string nome)
    {
        Nome = nome;
    }

    public Aluno(string nome, int idade)
    {
        Nome = nome;
        Idade = idade;
    }
}
```

This em Construtores

- Permite referenciar atributos da classe.
- Ajuda a evitar ambiguidades.
- Também pode chamar outro construtor.

Exemplo com this

```
class Cliente
{
    public string Nome;

    public Cliente(string Nome)
    {
        this.Nome = Nome;
    }
}
```

Chamando Outro Construtor

```
class Funcionario
{
    public string Nome;

    public string Cargo;

    public Funcionario() : this("Sem nome", "Sem cargo")
    {
    }

    public Funcionario(string nome, string cargo)
    {
        Nome = nome;
        Cargo = cargo;
    }
}
```

Inicialização Automática

- Atributos podem receber valores padrão.
- Tipos numéricos iniciam com 0.
- Boolean inicia com false.
- Objetos iniciam com null.

Valores Padrão em C#

- `int numero; // 0`
- `double preco; // 0.0`
- `bool ativo; // false`
- `string nome; // null`

Inicializadores de Objetos

```
Produto p = new Produto()  
{  
    Nome = "Mouse",  
    Preco = 150  
};
```

Boas Práticas com Construtores

- Inicialize atributos importantes.
- Evite lógica excessiva.
- Valide parâmetros recebidos.
- Use sobrecarga quando necessário.

Erros Comuns

- Esquecer de inicializar atributos.
- Criar construtores muito complexos.
- Duplicar lógica.
- Não validar parâmetros.

O que é um Destrutor?

- Método executado antes da remoção do objeto.
- Usado para liberar recursos.
- Executado automaticamente pelo Garbage Collector.

Sintaxe de um Destrutor

```
class Arquivo
{
    Arquivo()
    {
        Console.WriteLine("Objeto destruído");
    }
}
```

Características dos Destrutores

- Não possuem parâmetros.
- Não possuem modificadores de acesso.
- Não podem ser sobrecarregados.
- Pouco utilizados em aplicações modernas.

Garbage Collector

- Responsável pela limpeza de memória.
- Remove objetos não utilizados.
- Executa destrutores quando necessário.

lDisposale e using

- Alternativa moderna aos destrutores.
- Permite liberar recursos manualmente.
- Muito usado com arquivos e banco de dados.

Exemplo com using

```
using(StreamWriter sw = new  
StreamWriter("arquivo.txt"))  
{  
    sw.WriteLine("Exemplo");  
}
```

Comparação: Construtor x Destrutor

- Construtor: inicializa objetos.
- Destrutor: finaliza/libera recursos.
- Construtor é chamado na criação.
- Destrutor ocorre antes da remoção.

Desafio Final

- Criar sistema simples de cadastro.
- Aplicar construtores e destrutores.
- Utilizar inicialização automática.
- Apresentar funcionamento da aplicação.

Resumo da Aula

- Construtores inicializam objetos.
- Sobrecarga aumenta flexibilidade.
- Destrutores liberam recursos.
- Boas práticas tornam o código mais seguro.

Perguntas?

- Dúvidas e discussão final.